Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 Программная инженерия

Дисциплина «Сервис-ориентированная архитектура»

Отчет

По лабораторной работе №1

Вариант 8810

Выполнили:

Орчиков Даниил Валерьевич

Болорболд Аригуун

P3411

Преподаватель:

Кривоносов Егор Дмитриевич

Санкт-Петербург

2025

Вариант 8810

Текст задания ЛР

Разработать спецификацию в формате OpenAPI для набора веб-сервисов, реализующего следующую функциональность:

Первый веб-сервис должен осуществлять управление коллекцией объектов. В коллекции необходимо хранить объекты класса HumanBeing, описание которого приведено ниже:

```
public class HumanBeing {

   private Long id; //Поле не может быть null, Значение поля должно быть больше 0, Значение этого поля должно быть уникальным, Значение этого поля должно генерироваться автоматически

   private String name; //Поле не может быть null, Строка не может быть пустой

   private Coordinates coordinates; //Поле не может быть null

   private java.time.ZonedDateTime creationDate; //Поле не может быть null, Значение этого поля должно генерироваться автоматически

   private boolean realHero;

   private Boolean hasToothpick; //Поле не может быть null

   private int impactSpeed; //Максимальное значение поля: 58

   private WeaponType weaponType; //Поле может быть null

   private Mood mood; //Поле не может быть null

   private Car car; //Поле может быть null

}

public class Coordinates {

   private Integer x; //Значение поля должно быть больше -63, Поле не может быть null

   private double y;

}

public class Car {

   private Boolean cool; //Поле может быть null

}

public enum WeaponType {

   AXE,

   SHOTGUN,

   MACHINE_GUN;

}

public enum Mood {

   SADNESS,
```

SORROW,

GLOOM,

APATHY,

RAGE;

}

Веб-сервис должен удовлетворять следующим требованиям:

- API, реализуемый сервисом, должен соответствовать рекомендациям подхода RESTful.
- Необходимо реализовать следующий базовый набор операций с объектами коллекции: добавление нового элемента, получение элемента по ИД, обновление элемента, удаление элемента, получение массива элементов.
- Операция, выполняемая над объектом коллекции, должна определяться методом HTTP-запроса.
- Операция получения массива элементов должна поддерживать возможность сортировки и фильтрации по любой комбинации полей класса, а также возможность постраничного вывода результатов выборки с указанием размера и порядкового номера выводимой страницы.
- Все параметры, необходимые для выполнения операции, должны передаваться в URL запроса.
- Информация об объектах коллекции должна передаваться в формате **json**.
- В случае передачи сервису данных, нарушающих заданные на уровне класса ограничения целостности, сервис должен возвращать код ответа http, соответствующий произошедшей ошибке.

Помимо базового набора, веб-сервис должен поддерживать следующие операции над объектами коллекции:

- Вернуть массив объектов, значение поля name которых содержит заданную подстроку.
- Вернуть массив объектов, значение поля impactSpeed которых меньше заданного.
- Вернуть массив уникальных значений поля impactSpeed по всем объектам.

Эти операции должны размещаться на отдельных URL.

Второй веб-сервис должен располагаться на URL /heroes, и реализовывать ряд дополнительных операций, связанных с вызовом API первого сервиса:

- /search/{real-hero-only} : найти всех героев (или только реальных)
- /team/{team-id}/car/add : пересадить всех героев команды, не имеющих автомобиля на красные "Lada Kalina"

Эти операции также должны размещаться на отдельных URL.

Для разработанной спецификации необходимо сгенерировать интерактивную веб-документацию с помощью Swagger UI. Документация должна содержать описание всех REST API обоих сервисов с текстовым описанием функциональности каждой операции. Созданную веб-документацию необходимо развернуть на сервере helios.

Репозиторий с кодом

Инструкции по настройке, деплою и развертыванию вашей лабораторной работы

Скачиваем swagger ui

SWAGGER_VER=5.17.14

wget https://github.com/swagger-api/swagger-ui/archive/refs/tags/v$SWAGGER_VER.zip

unzip v$SWAGGER_VER.zip

cp -r swagger-ui-$SWAGGER_VER/dist/* ~/public_html/soa/swagger/

rm -rf swagger-ui-$SWAGGER_VER v$SWAGGER_VER.zip

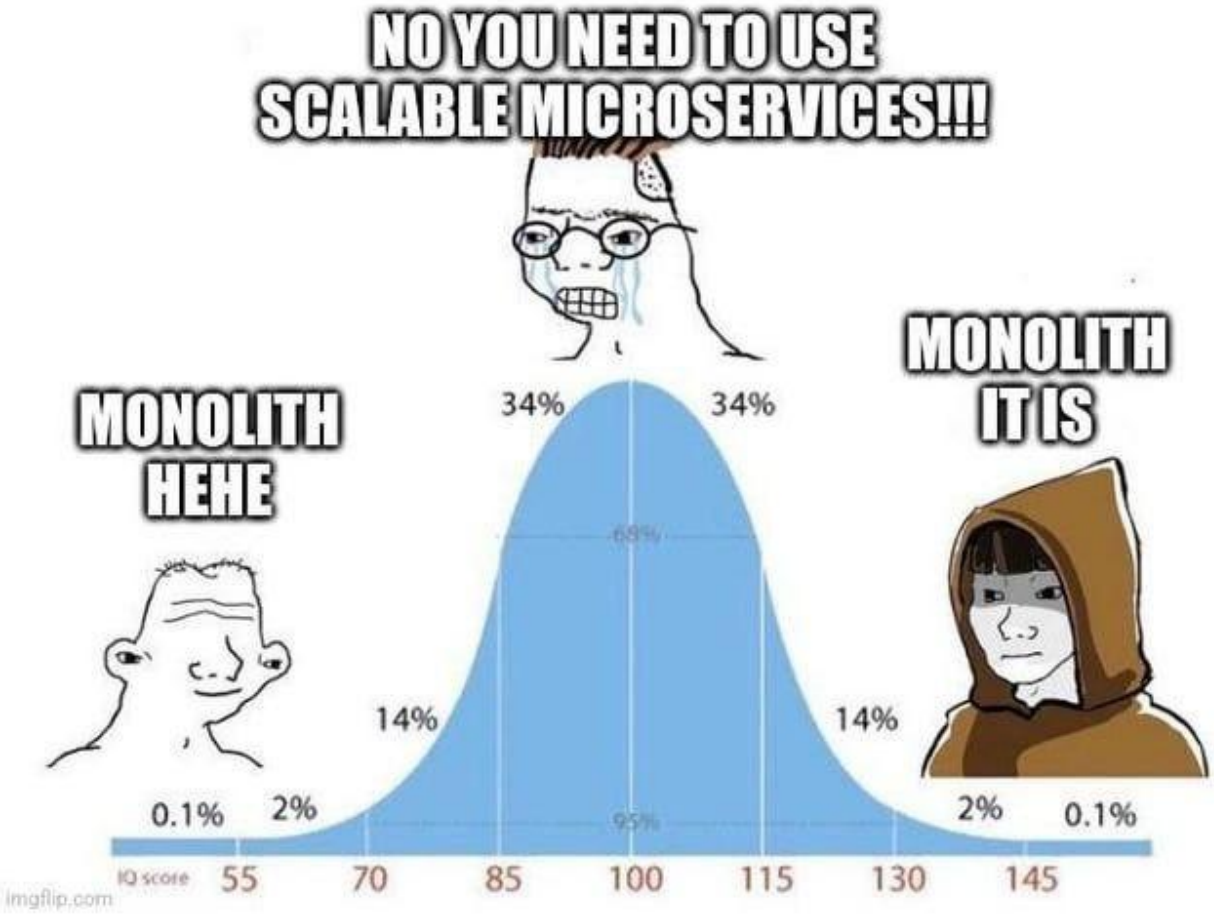
cd ~/public_html/soa/swagger/
Перемещаем сюда файлы HeroService.yaml и HumanBeingService.yaml

В файле swagger-initializer.js меняем url: … на:

```
urls: [
    { url: "HeroService.yaml",  name: "HeroService" },
    { url: "HumanBeingService.yaml", name: "HumanBeingService" }
],
"urls.primaryName": "HeroService",
```

Вывод

В рамке этой работы мы создали спецификацию OpenAPI 3.1.0, ручки и функциональности которой были заранее выдан вариантом. По мере создания спецификации мы убедились в нужности чётких, недвусмысленных определении каждого функционала, запроса, параметра и ответа, который соответствует принципам идеологии СОА. С помощью созданной спецификации появится возможность реализации сервисов, удовлетворяемые с точки зрения точности, корректности архитектуры и дальнейшей масштабируемости.


NO YOU NEED TO USE SCALABLE MICROSERVICES!!!
MONOLITH HEHE
MONOLITH IT IS
34%
34%
68%
14%
14%
0.1%
2%
95%
2%
0.1%
IQ score
55
70
85
100
115
130
145
imgflip.com